DAY 2 OF 5 · CONCEPTS PRIMER

Everything You Need to Know to Do the Four Day 2 Tasks

A plain-language explanation of every architecture, data and security idea the Day 2 hands-on tasks depend on — with a picture for each one. No prior experience of production systems required.

> **Who this is for.** If a phrase in the task pack — *anomaly detection*, *orchestration layer*, *feature store*, *embedding version*, *access-filtered retrieval*, *idempotency* — is unfamiliar, this document explains it before you need it. If all of them are familiar, use the cheat sheet at the back and skip the rest.

Part	Supports	Concepts covered
Part 0	All four tasks	Why architecture is the decision with the longest half-life
Part 1	Task 1 — Pattern selection	The thirteen patterns, the twelve criteria, the six-question sequence, predictive ML versus anomaly detection, why scarce labels change everything, RAG versus fine-tuning versus prompting, agent reliability, deployment shape, anti-patterns
Part 2	Task 2 — The nineteen layers	What a production system has that a notebook does not, the access layers, orchestration, retries and timeouts, model serving, the registries, and the four observability layers
Part 3	Task 3 — Ingestion architecture	The eleven storage layers, the five stores, embedding versions, ingestion patterns, data contracts, lineage and reproducibility, quality checks, training–serving skew
Part 4	Task 4 — Secure retrieval	What is different about securing an AI system, identity and RBAC, access-filtered retrieval, per-role evaluation, secure tool calling, and the four leakage paths
Appendix	All four tasks	Checklists you can apply directly, and a glossary

> **How to read a section.** Each concept has the same three parts: **what it is** in one or two sentences, **a picture**, and **why the task needs it**. If you are short of time, read the figures and their captions — they carry most of the content on their own.

Part 0 — Why today is about architecture

Read this even if you read nothing else

The decision with the longest half-life

Yesterday established that most stalled proofs of concept are limited by data, labels or measurement rather than by the model. Today addresses the next question: given what the problem actually is, **is the architecture you chose the right one** — and does it have the layers a production system requires?

The distinction that makes architecture different from every other decision you will make this week: a wrong metric can be replaced in an afternoon. A wrong architecture is not improved by better prompts or a larger model. It is **rewritten**.

Most architectures were never chosen

They were inherited — from a tutorial, from the last project, or from whatever the first engineer happened to be comfortable with. Six recognisable ways this happens:

Failure mode	What it looks like
Pattern by habit	The last project used retrieval, so this one does too — even though the problem is time-series anomaly detection with no documents in it.
Pattern by hype	Agents were chosen because agents are current, not because the task needs autonomy. Complexity arrives with no corresponding capability.
Pattern by demo	It worked in a notebook, so the notebook became the architecture. Nothing needed orchestrating with one user.
Pattern by vendor	A platform was purchased and the problem reshaped to fit it. Sometimes reasonable, frequently expensive, rarely examined.
No pattern at all	Components accumulated as requirements arrived. Nobody can draw the system, which means nobody can reason about it.
Right pattern, missing layers	The pattern is correct, but authentication, audit, monitoring and feedback were never added — so it cannot be deployed.

The last row is the most common of the six among competent teams, and it is what Task 2 exists to fix.

What "architecture" means for the rest of today

Not a diagram. An architecture is a set of claims about how a system behaves — under load, under failure, under an access rule. Every task today ends with those claims tested rather than drawn, because a claim you have not tested is a picture.

> **A note on notation.** Today needs almost no mathematics. Where numbers appear they are counts, percentages and durations. What today does need is precision about a few words that get used loosely: *authentication* is not *authorisation*; *logging* is not *audit*; a *workflow* is not an *agent*; and an *embedding* is not a number, it is a number **relative to a particular model**. Each of those distinctions has a section below.

Part 1 — Concepts for Task 1

1.1 The selection sequence

There are twelve selection criteria, and applying all twelve to every candidate pattern is slow and confusing. Applying six of them **in order** is fast, because the early questions eliminate whole families of pattern before you have to think about the later ones.

Figure 1 · The selection sequence — answer these in order, not all at once

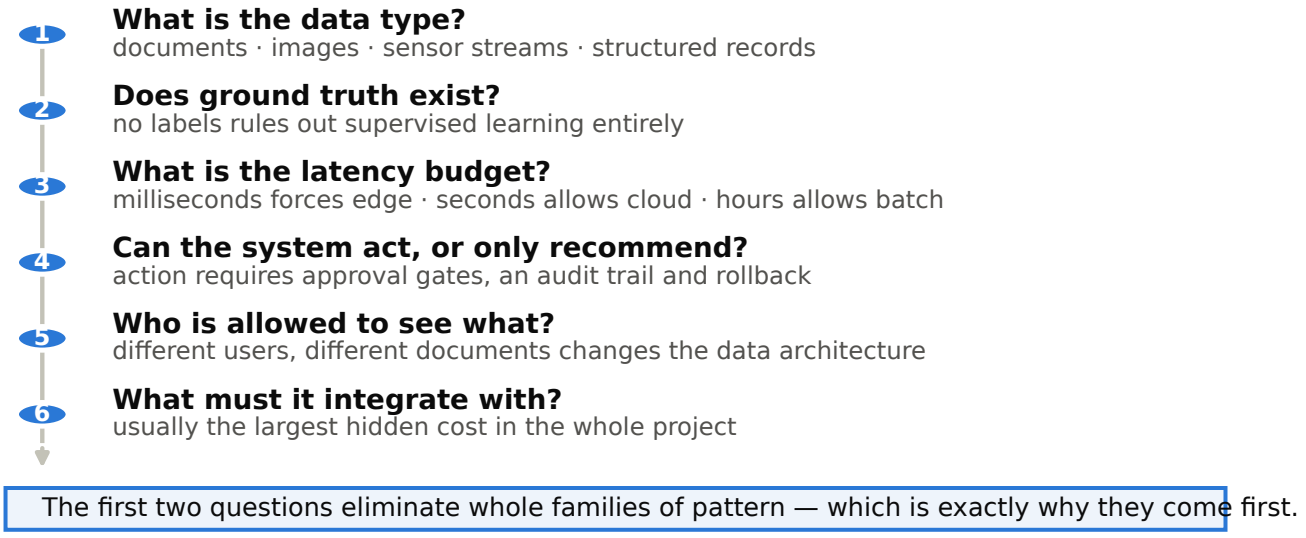

Figure 1. Data type alone rules out most of the thirteen patterns. Ground truth availability rules out supervised learning entirely when labels do not exist — which is the situation in Task 1, and in most industrial predictive-maintenance projects. Only after those two is it worth discussing latency, autonomy, access and integration.

The other six criteria — accuracy target, explainability, update frequency, drift risk, cost constraint, and deployment location — matter, but they refine a choice rather than making one.

1.2 Predictive ML and anomaly detection — the same problem, two shapes

These are the two candidates for almost every industrial sensor problem, and the choice between them is decided by one question: **do you have labelled failures, and enough of them?**

Figure 2 · Two patterns for the same industrial problem — and one of them needs failures to be labelled

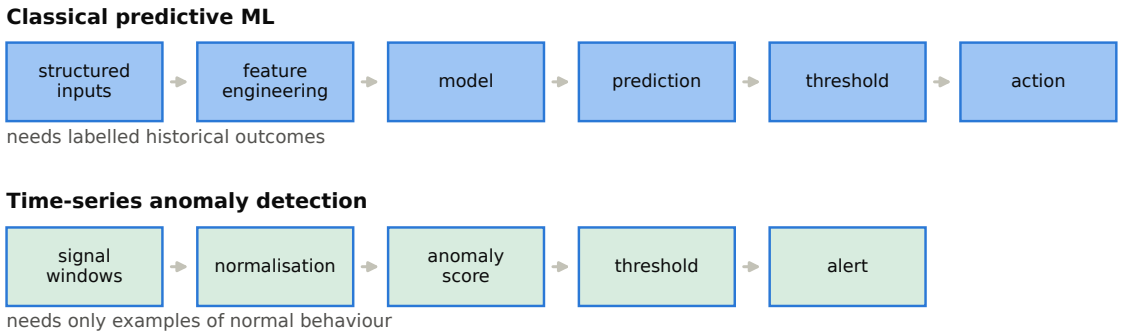

Figure 2. Predictive ML learns the boundary between "will fail" and "will not" from labelled historical outcomes. Anomaly detection learns what normal looks like and flags departures from it — so it needs examples of normal operation, which every monitored asset produces continuously, and no examples of failure at all.

Design decision	Predictive ML	Anomaly detection
Labels needed	Historical outcomes required	Normal behaviour is enough
Handles rare failures	Poorly — severe class imbalance	Well — that is the design intent
Explains why	Feature importance, SHAP values	Which signal deviated, and by how much
Main risk	Temporal and group leakage in features	Alert fatigue from a poorly set threshold

> **The threshold work from Day 1 applies unchanged.** An anomaly score needs a cost-optimal cutoff exactly as a classifier does, and the alerts-per-engineer-per-day arithmetic is identical. Anomaly detection changes what you need in order to *train*; it changes nothing about what you need in order to *operate*.

1.3 Why eleven labelled failures is not a small dataset — it is a different problem

The instinct with few labels is that the model will simply be less accurate. The real consequence is worse and less obvious: **you lose the ability to measure**. And a system you cannot measure is one you can never improve, which is exactly the position Day 1 spent eight hours diagnosing.

Figure 3 · With a handful of labelled failures you cannot measure recall at all

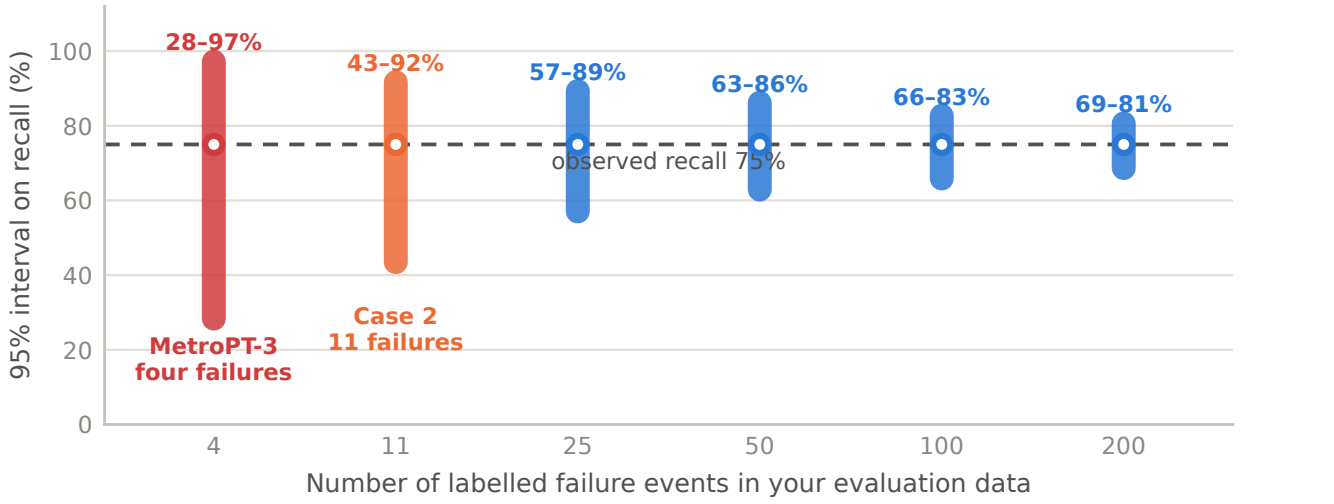

Figure 3. Suppose your model catches three failures out of four. That is a recall of 75% — with a 95% interval running from roughly 28% to 97%. You cannot distinguish a good model from a poor one, you cannot demonstrate an improvement, and you cannot fail a model that deserves to fail. Even at eleven events the interval spans half the range.

> **How to say it.** "If you have three years of data and eleven labelled failures, a supervised classifier is not the honest choice. You do not have enough positive examples to learn from, and — more decisively — not enough to *evaluate against*. Anomaly detection asks a question your data can actually answer: is this behaviour unusual for this asset? And it does not need failures to be labelled at all."

1.4 RAG, fine-tuning and prompting

The most misunderstood choice of the day, and the one with the largest cost of getting wrong. The three do genuinely different things, and only one of them adds knowledge.

Figure 4 · Three different things, routinely confused

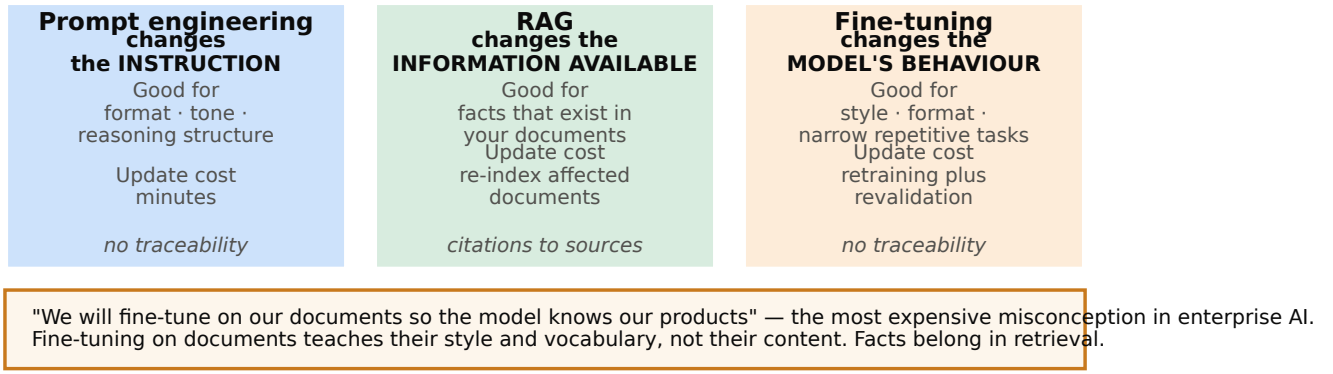

Figure 4. Prompting changes the instruction. Retrieval changes the information available. Fine-tuning changes the model's behaviour. Notice which column carries citations — traceability is an architectural property, not a feature you can add later.

> **The new engineer.** Prompting is giving them clearer instructions. Retrieval is handing them the manual. Fine-tuning is sending them on a training course that changes how they work. If they simply do not know your equipment, the training course will not help — they need the manual. Fine-tuning to add facts is sending someone on a course to memorise a document they could have been given.

When fine-tuning genuinely is the right answer — three cases, and none of them is knowledge. A fixed output format the model keeps deviating from despite prompting. A narrow, high-volume, repetitive task where a smaller fine-tuned model is dramatically cheaper per call. Domain language so specialised that the base model misreads the terminology. All three should be preceded by a demonstration that prompting and retrieval could not achieve it.

1.5 Agents, workflows, and why autonomy multiplies error

An agentic system decides its own sequence of actions. That is a real capability and a real liability, and the arithmetic decides which one you are buying.

Figure 5 · Autonomy multiplies error rather than adding it

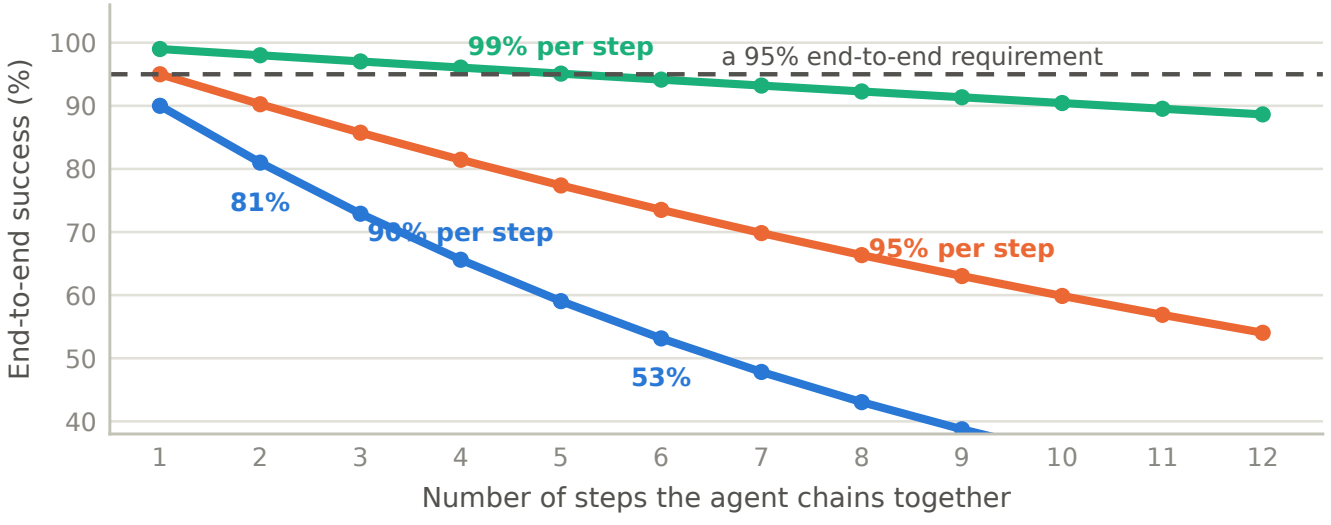

Figure 5. A step that succeeds 90% of the time succeeds 81% over two steps and 53% over six. Autonomy multiplies error rather than adding it. This is why an agentic system needs guardrails, approval gates and per-step observability far more than it needs more autonomy.

Use…	When	Because
An agent	The steps genuinely cannot be enumerated in advance, several tools must be chosen dynamically, and the cost of a wrong sequence is recoverable	You are buying flexibility and paying for it in reliability
A workflow	The steps are known, even if there are many of them	A directed pipeline is cheaper, faster, testable, debuggable and predictable

The question that settles it. Can you list the steps? If yes, build the workflow. Most problems described as "agentic" are workflows with one or two conditional branches — and you get the same outcome with a fraction of the cost, latency and failure surface, plus the ability to actually test it.

1.6 Deployment shape — four decisions that constrain everything else

Inference timing (batch or real-time), agent structure (single or multiple), deployment location (cloud, on-premises, edge), and model hosting (managed API or self-hosted). The first is decided almost entirely by the latency budget.

Figure 6 · The latency budget picks the deployment shape, not the other way round

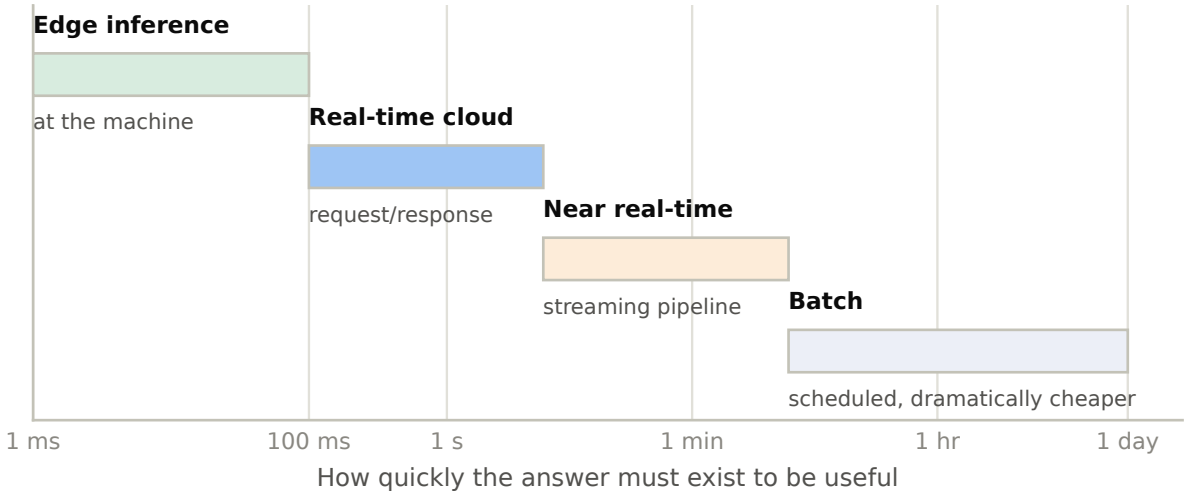

Figure 6. How quickly the answer must exist to be useful picks the deployment shape. Milliseconds forces inference at the machine. Hours allows batch, which is dramatically cheaper and simpler than anything above it — and is frequently rejected for no reason other than that it sounds less impressive.

The question that resolves most of these at once. "What happens when the network is down for four hours?" If the answer is that the line stops, you need edge inference and a local fallback. If the answer is that scoring catches up overnight, batch in the cloud is fine. One sentence eliminates several options, and it is far more decisive than a general preference for one deployment model.

For industrial vision specifically, note that **where inference runs** is a harder decision to change later than the model architecture — it determines latency, bandwidth cost, model size, update mechanism, failure behaviour when the network drops, and what data ever leaves the plant. It is usually decided by accident.

1.7 Three patterns that combine well

Pattern	Core idea	The trade-off nobody mentions
Edge AI	Inference runs at the machine	Sub-100 ms decisions and no network dependency — at the cost of constrained model size, harder updates, fleet management, and physical access to debug
Human-in-the-loop	The system defers rather than guesses	Not a fallback — an architectural component. It needs a queue, a reviewer interface, routing logic, service expectations, and a feedback path into training
Digital twin	Simulation generates what reality cannot	Produces training data for rare failures — and risks the model learning the simulator's quirks rather than the physics

Edge inference with a plant-server review queue, trained partly on simulated rare failures, is a coherent industrial target architecture and worth naming as one.

1.8 Six anti-patterns — competent teams building the wrong thing well

1. **Retrieval for a problem with no documents.** The task is anomaly detection over sensor data. Retrieval adds latency, cost and a new failure mode, and answers nothing.
2. **Fine-tuning to add facts.** Fluent, confident, wrong answers about the new domain — with no citation to check.
3. **An agent where a workflow would do.** Cost, latency and failure surface, bought without capability.
4. **Real-time where batch would do.** Complexity and infrastructure spend for an answer nobody reads until morning.
5. **No human-in-the-loop on a high-cost decision.** The system acts confidently on the cases it understands least.
6. **One system serving several distinct problems.** Every requirement pulls the architecture in a different direction and none is served well.

Part 2 — Concepts for Task 2

2.1 A proof of concept has three components. A production system has nineteen.

The other sixteen are not bureaucracy. Each one exists because something failed without it.

Figure 7 · The gap you are being asked to close

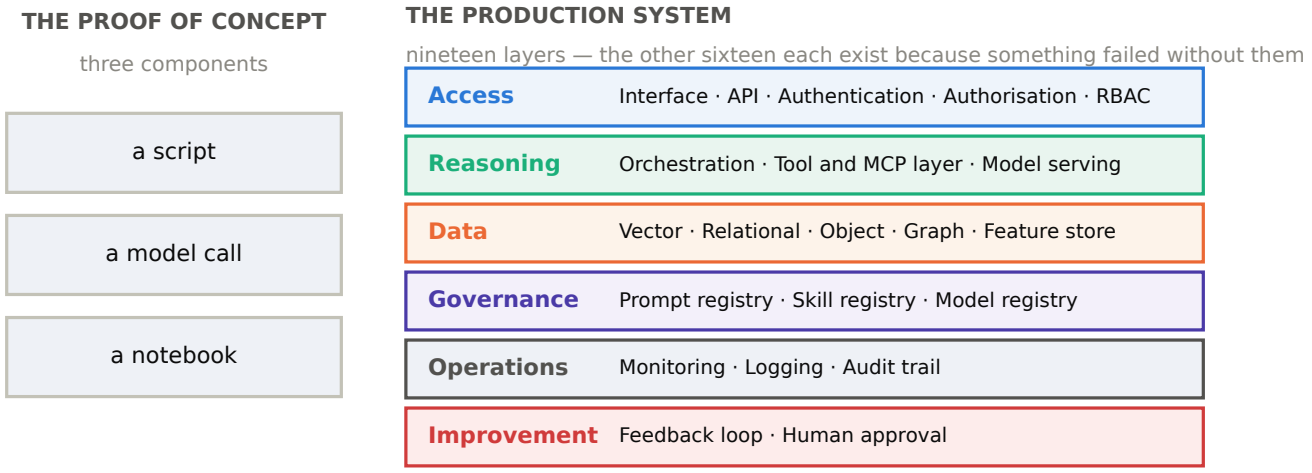

Figure 7. The six groups and what breaks without each. Access: anyone can see anything, with no accountability for any request. Reasoning: no retries, no fallbacks, no version control over behaviour. Data: sprawl, no lineage, no way to reproduce a result. Governance: nobody knows which version produced which answer. Operations: silent degradation, and no evidence when challenged. Improvement: the system cannot improve and cannot be trusted with action.

> **The prototype in the workshop.** A working prototype on the bench needs the machine and a power lead. The same machine on a production line needs guarding, interlocks, isolation, maintenance access, logging, spares, an operating procedure and someone trained to run it. Nobody calls those requirements bureaucracy. They are what makes it a production machine rather than a demonstration.

> **The caveat that matters.** Not every system needs all nineteen layers on day one. Every system needs a **deliberate decision** about each — including the decision to leave one out for now, and a note of why. The gap between "we decided not to" and "we never thought about it" is the whole difference, and it is why Task 2 scores exclusions.

2.2 Authentication is not authorisation

These two words are used interchangeably in conversation and mean entirely different things in a system. Confusing them is the root of the most common enterprise AI security failure.

Layer	Answers	In practice
User layer	Where is the answer consumed?	The best AI systems are invisible — they appear inside the tool the engineer already uses, not in a new portal nobody opens
API layer	How does anything reach the system?	One controlled entry point: rate limiting, request validation, versioning, timeouts, a stable contract. Without it, every consumer couples to your internals and you can never change them
Authentication	**Who is this?**	Federated to the enterprise identity provider — never a separate credential store for the AI system
Authorisation	**What may they do?**	A user may be legitimately authenticated and still not permitted to invoke a given tool
RBAC	Which documents, tools and actions per role?	For AI systems this must reach *into retrieval itself* — see Part 4

> **The most common enterprise AI security failure, stated precisely.** Authentication enforced at the API, and retrieval that then searches the entire corpus regardless of who asked. The user is correctly identified and then served content they should never see. Part 4 resolves it.

2.3 The orchestration layer — where reliability lives

In a proof of concept, orchestration is a script. In production it is the component that turns a model call into a system, and it has six responsibilities.

Figure 8 · What the orchestration layer actually does to one request

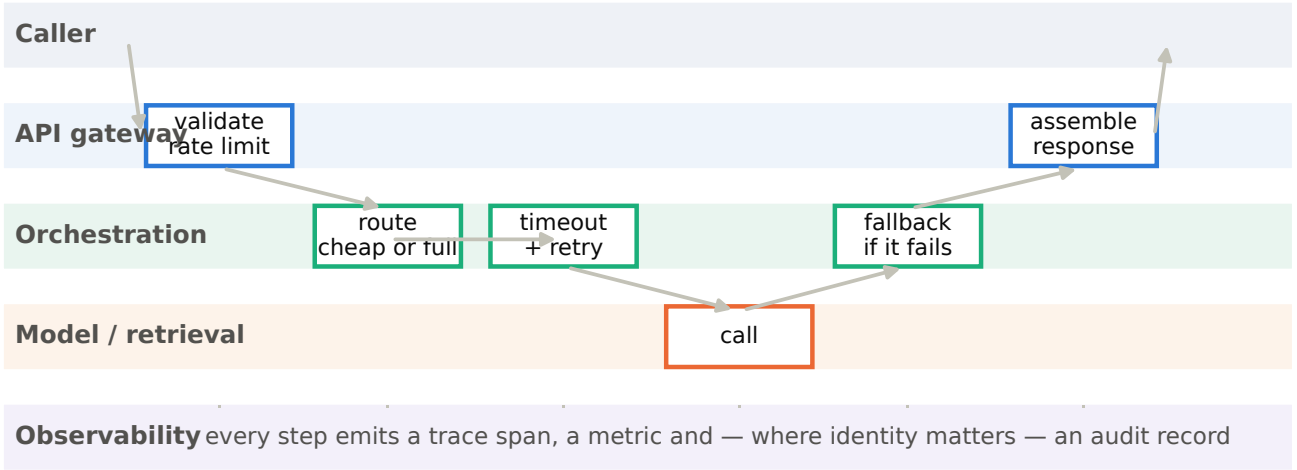

Figure 8. One request through the layers. Routing decides which model, prompt or retrieval path handles it — enabling cheap models for easy requests. Sequencing makes each step separately observable and testable. Retry and fallback handle the failures that are normal at scale. Timeouts stop a slow dependency becoming a system-wide outage. Context assembly builds what the model actually sees. Guardrails run here, not inside the prompt.

> **How to say it.** "Your proof of concept has no retries, no timeouts and no fallbacks — and it does not need them, because there is one user who will simply run it again. At a thousand requests an hour, a dependency that fails one time in five hundred fails twice an hour, and without an orchestration layer each of those is a user-visible error."

2.4 Retry and timeout are not the same control, and one without the other is worse than neither

Retry handles a transient failure. A timeout bounds how long you wait. Teams add the first and forget the second, and the result actively harms availability.

Figure 9 · Retry without a timeout does not add resilience — it multiplies the outage

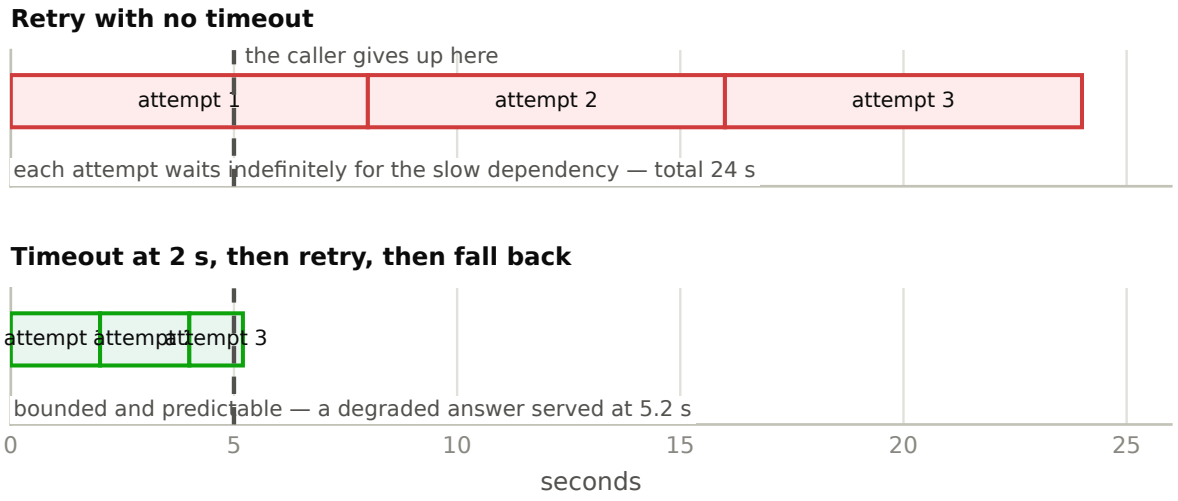

Figure 9. With no timeout, each retry waits indefinitely on the slow dependency — so three attempts take three times as long, and the caller has already given up. With a timeout, the total is bounded and predictable, and a degraded answer arrives inside the caller's patience. Every external call needs a deadline, and the deadline must be shorter than your own caller's.

2.5 Model serving, tools, and why the tool layer is architecture

Model serving handles versioning so every response is traceable, scaling and concurrency limits, caching so identical requests are not recomputed, cost control, fallback, and isolation between consumers.

The **tool and MCP layer** handles tool contracts, access boundaries, call logging, failure semantics, rate limits and idempotency.

> **Why the tool layer is architecture rather than plumbing.** The moment your system can call a tool, it can take an action with consequences — order a part, close a ticket, dispatch an engineer. Every property that makes actions safe (authorisation, logging, idempotency, approval gates, rollback) has to live in this layer, because **the model cannot be relied upon to enforce any of them itself**.

2.6 The three registries

Version control for the things that determine behaviour but are not code.

Registry	Why it exists
Prompt registry	Prompts change behaviour as much as code does, and are usually edited by whoever is closest to the problem. Versioned, reviewed, tested, and linked to the evaluation result they produced.
Skill registry	Reusable capability packages — instructions, schemas, tool bindings, safety rules, test cases. Prevents twelve teams writing twelve versions of the same troubleshooting routine.
Model registry	Which version is serving, what it scored, on which dataset version, approved by whom. The prerequisite for rollback and the foundation of every MLOps practice.

> **The question that makes registries concrete.** "Six months from now, an engineer challenges an answer your system gave. Can you reconstruct exactly what produced it — which model, which prompt version, which documents, which retrieval settings?" Without registries the answer is no. In a safety-relevant or regulated context that is a serious problem rather than an inconvenience.

2.7 Monitoring, logging, audit and feedback are four different things

Figure 10 · Four different jobs. Teams routinely try to do all four from the application log.

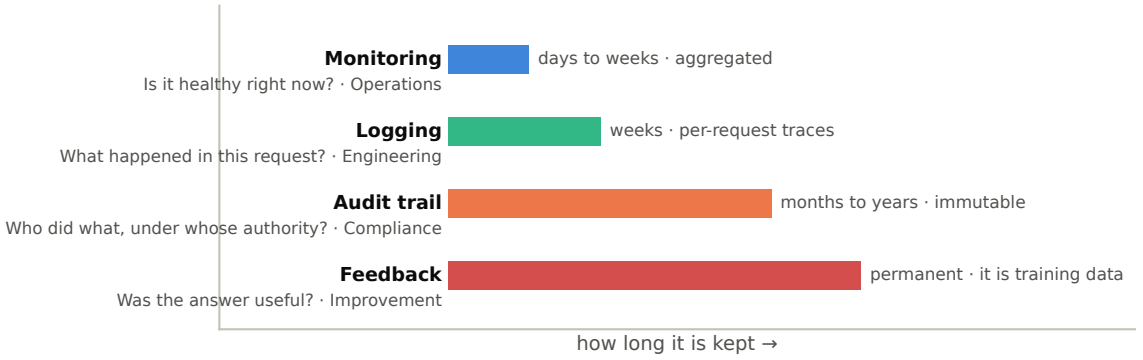

Figure 10. Different purposes, different audiences, different retention, different shapes. Monitoring needs aggregation and speed. Audit needs immutability and long retention. Feedback needs to be joinable to training data. Designing them together at the start is far cheaper than separating them after the first compliance review.

> **Watch for.** Teams routinely try to serve all four purposes from application logs. It works until the first time someone asks a question that needs one of the other three — and by then the data has the wrong shape and the wrong retention.

Part 3 — Concepts for Task 3

3.1 The eleven storage layers

A single data lake is not an answer. Different objects have genuinely different requirements for versioning, mutability, retention and access — and putting them in one place means every one of those requirements is wrong for most of the contents.

Figure 11 · The eleven storage layers, and the property that defines each

If you cannot say which of these eleven a given object belongs to, it is probably in three of them, inconsistently — and one of the three is somebody's laptop.

01 Raw data	source data exactly as received	*immutable — your ability to reprocess*
02 Cleaned data	validated, deduplicated, schema-conformant	*reproducible from raw*
03 Feature layer	computed features for model input	*identical in training and serving*
04 Embedding layer	vector representations	*versioned with the model that made them*
05 Vector index	searchable structure over embeddings	*rebuildable, filterable metadata*
06 Metadata store	source, version, access level, dates	*what makes filtered retrieval possible*
07 Ground truth	verified correct answers and labels	*versioned, provenance per label*
08 Feedback store	ratings, corrections, escalations	*joinable to the original request*
09 Audit store	access records	*immutable, long retention*
10 Model artifacts	weights and configuration	*versioned with the data behind them*
11 Prompts & policy	prompts, guardrails, Skill definitions	*reviewed and approved like code*

Figure 11. Each layer with the property that defines it. Raw data is immutable because it is your ability to reprocess. Cleaned data must be reproducible from raw by a versioned transformation. Features must be computed identically in training and serving. Audit records must never be rewritten.

3.2 The five stores, and the questions they answer

Figure 12 · Five stores, five questions. Using one store for everything is the most common data mistake in AI systems.

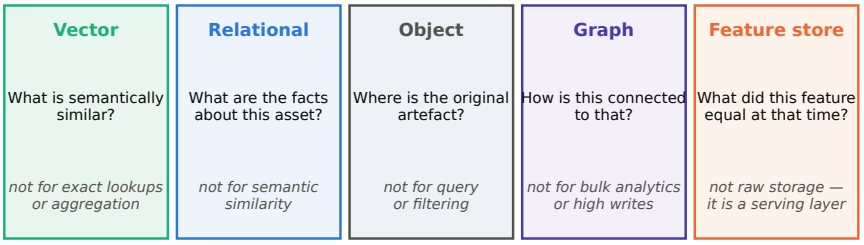

Figure 12. Each store answers one kind of question well and others badly. A vector database is superb at "what is semantically similar" and unusable for exact lookups, aggregation or transactions. Using one store for everything is the most common data architecture mistake in AI systems.

3.3 An embedding is only meaningful relative to the model that produced it

This is the single most important sentence in Part 3, and the failure it prevents is expensive, common, and completely silent.

An embedding turns text into a list of numbers positioned in a space that a *particular model* constructed. Two models can produce lists of the same length whose numbers mean entirely different things. Distances between them are meaningless — and because the lengths match, nothing errors.

Figure 13 · Two embedding versions in one index — the failure nothing reports

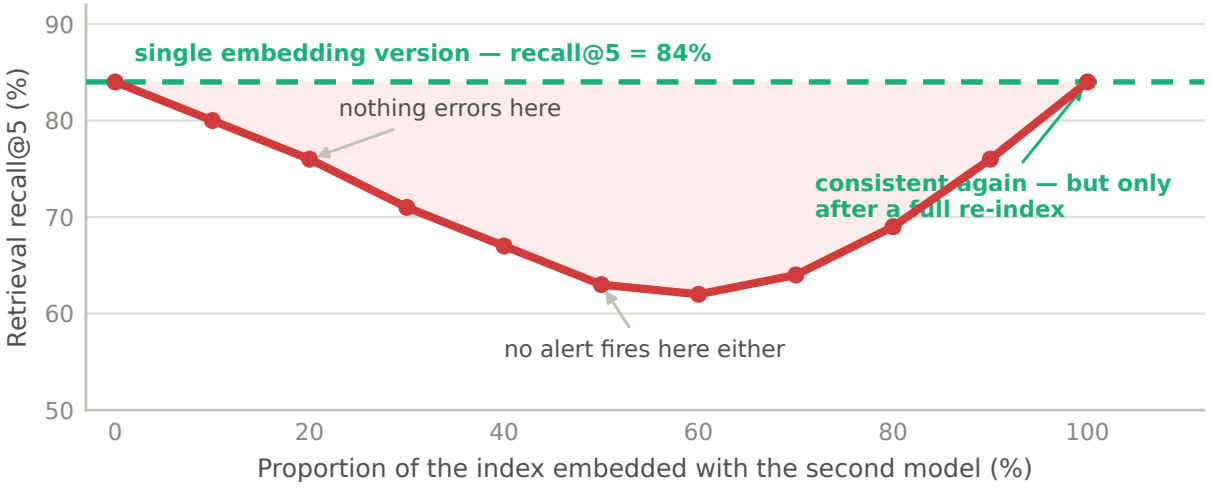

Figure 13. A newer embedding model is released and used for new content, added to the existing index. Retrieval quality falls as the mixture grows, recovers only when the index is consistent again, and at no point does anything raise an exception or fire an alert. Diagnosing this months later is genuinely difficult, because the symptom is diffuse — some questions get worse, most are fine, and nothing in the logs changed.

> **The architectural rule.** Record the embedding model version as metadata on every vector, and treat an embedding-model change as a **full re-index** — budgeted in time and cost like the migration it is. Either re-embed everything, or maintain a separate index per version.

3.4 Ingestion patterns — chosen by staleness, not by technology

Figure 14 · The question that chooses the ingestion pattern is about staleness, not technology

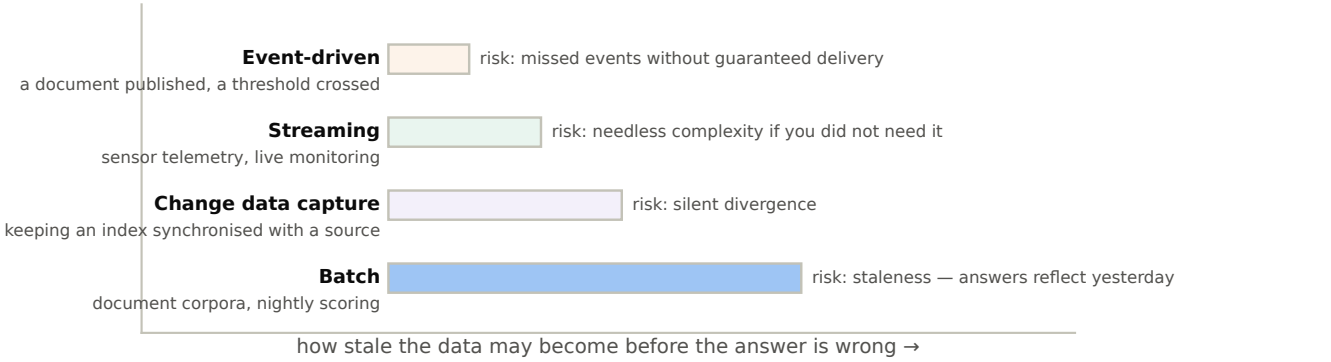

Figure 14. Four patterns, ordered by how stale the data is allowed to become. Batch is dramatically simpler and cheaper and is correct far more often than teams assume. Event-driven costs more and is the only honest answer when a stale answer is actively harmful.

> **The question that chooses between them.** "How stale may this data be before the answer becomes wrong?" For service manuals, nightly batch is fine. For vibration monitoring, minutes matter. For a procedure that has just been superseded by a safety notice, you need event-driven ingestion — because a system confidently citing a withdrawn procedure is considerably worse than one that finds nothing.

3.5 Data contracts — turning a silent failure into a loud one

A data contract is an agreement between the producer of data and its consumer, **enforced automatically rather than assumed**. It specifies schema, semantics, quality guarantees, change policy, ownership and service expectation.

Figure 15 · A rejected batch is a bad morning. A quarter of quietly degraded answers is a bad quarter.

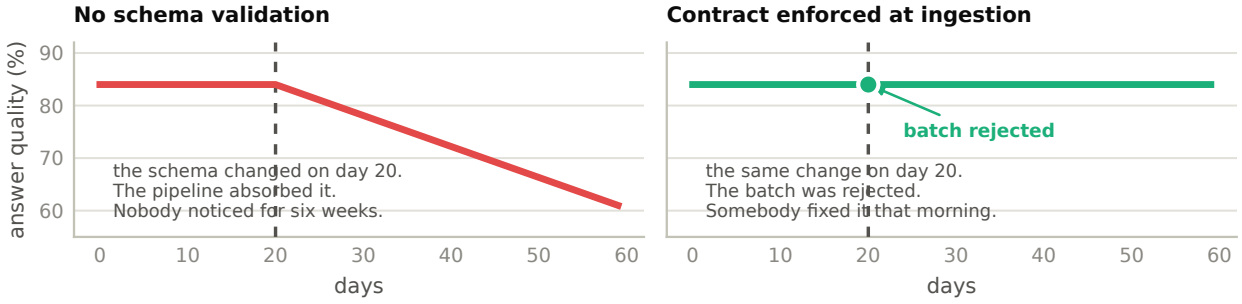

Figure 15. A traditional pipeline fails loudly when a schema changes. An AI system absorbs the malformed input, produces a confident and wrong answer, and nobody notices for weeks. Validation at ingestion converts a silent quality failure into a loud pipeline failure — which is exactly the trade you want.

Eight quality checks and where each runs: completeness, validity (ingestion) · consistency, uniqueness, referential integrity (cleaning) · freshness, distribution (continuous) · label quality (periodic). The distribution check is covariate-shift detection from Day 1, and it is the only drift signal available without waiting for labels.

3.6 Training–serving skew, and the component that exists to prevent it

A feature is computed one way in the training pipeline — typically in a notebook, over a full historical dataset — and another way in production, typically in application code, over a single record. The definitions drift apart, and the model performs worse in production than in testing with no visible cause.

Figure 16 · Training-serving skew, and the one component that exists to prevent it

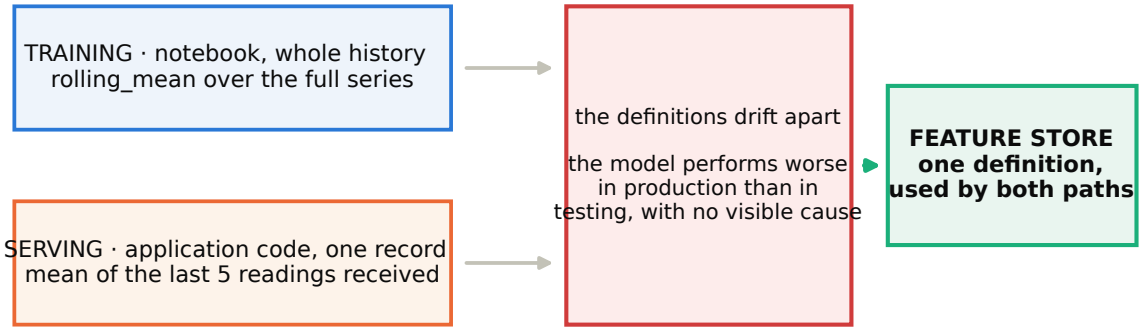

Figure 16. A feature store exists to solve exactly this one bug: one definition, used by both paths. If a team has ever seen unexplained production underperformance, this is a leading candidate.

3.7 Lineage and reproducibility

Five related capabilities — data lineage, data versioning, transformation versioning, model-to-data binding, and prompt and policy versioning — that together answer one question: **can you reconstruct how this answer came to exist?**

> **The test to run on yourself.** Pick a result from three months ago and reproduce it exactly: same data version, same feature code, same model, same prompt. Most teams cannot. Discovering that in a workshop is considerably cheaper than discovering it in an audit.

3.8 Retention, masking and where to apply them

- **Retention.** Raw data long, for reprocessing. Audit long, for compliance. Personal data only as long as lawful. Different layers need genuinely different policies; a single blanket rule will be wrong for most of them.
- **Masking.** Mask at ingestion, where the AI system never needs the value. Masking at output is far weaker, because the value has already entered the context.

- **Access-controlled retrieval.** Filter at the query, not after — covered fully in Part 4.

> **The distinction worth remembering.** Once sensitive data is in the model's context, you are relying on the model not to repeat it. That is not a control — it is a hope. Mask before it gets there.

Part 4 — Concepts for Task 4

4.1 Six assumptions a traditional security review makes, and which AI breaks

AI systems are not generally riskier than other software. They break six specific assumptions, and every control below exists because one of those six was broken.

Traditional application	AI system	Consequence
Returns records the query selected	Returns synthesised text from many sources	Access control must apply **before** synthesis
Input is structured and validated	Input is free-text instruction	Instructions can be injected through input
Behaviour defined by code	Behaviour defined by prompt, model and context	The attack surface includes data the model reads
Actions are explicit code paths	Actions are chosen by the model at runtime	Every reachable tool is a potential unintended action
Data flows are enumerable	Retrieval assembles context dynamically	Leakage paths are harder to enumerate and test
Failure is an error	Failure is a plausible, confident, wrong answer	**Security failures are silent**

4.2 Secure retrieval — the control that matters most

This is the AI-specific security problem. Get it wrong and every other control is decoration.

Figure 17 · The control that matters most — and the order of two steps is the whole difference

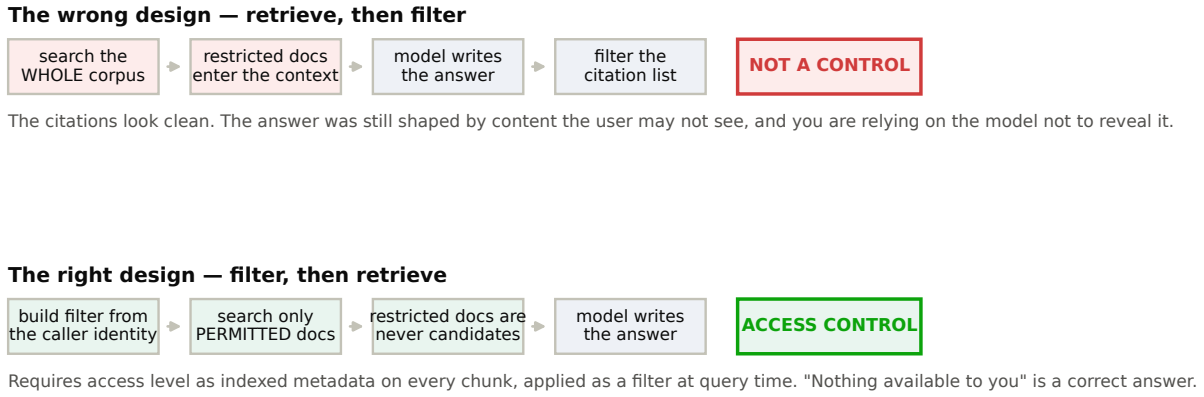

Figure 17. The same components in a different order, with completely different security properties. In the upper design the restricted content has already entered the model's context and shaped the answer — filtering the citation list afterwards changes what the user is shown, not what the answer was built from.

> **The librarian.** A librarian who fetches every book, reads them, summarises them and then tries not to mention the restricted ones has already read the restricted ones to you, in aggregate. A librarian who only searches the shelves you are permitted to use has not. Only the second is access control.

Two consequences follow, and almost no team has considered either. First, **"no relevant documents found" is a correct answer** for a user who lacks access, and the system must handle that path gracefully rather than treating it as a failure to work around. Second, evaluation must run per role.

4.3 Per-role evaluation

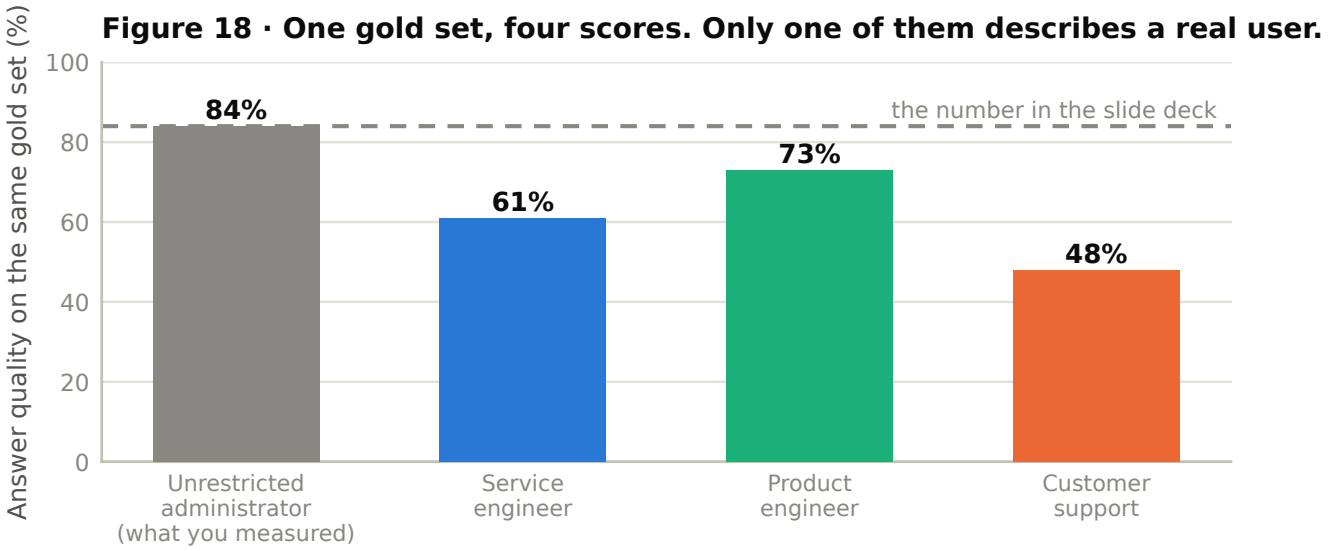

Figure 18. A gold set built with unrestricted access measures an experience no real user has. Each role's answer quality is bounded by what that role can reach, and the differences between the three are the honest picture of what each population actually receives.

4.4 RBAC, ABAC, and why to start simple

Identity federated to the enterprise provider; the AI system never holds its own credential store; identity propagates all the way into retrieval and tool calls. **RBAC** attaches permissions to roles — simple, auditable, the right default. **ABAC** computes permissions from attributes of user, resource and context — more expressive, and much harder to test.

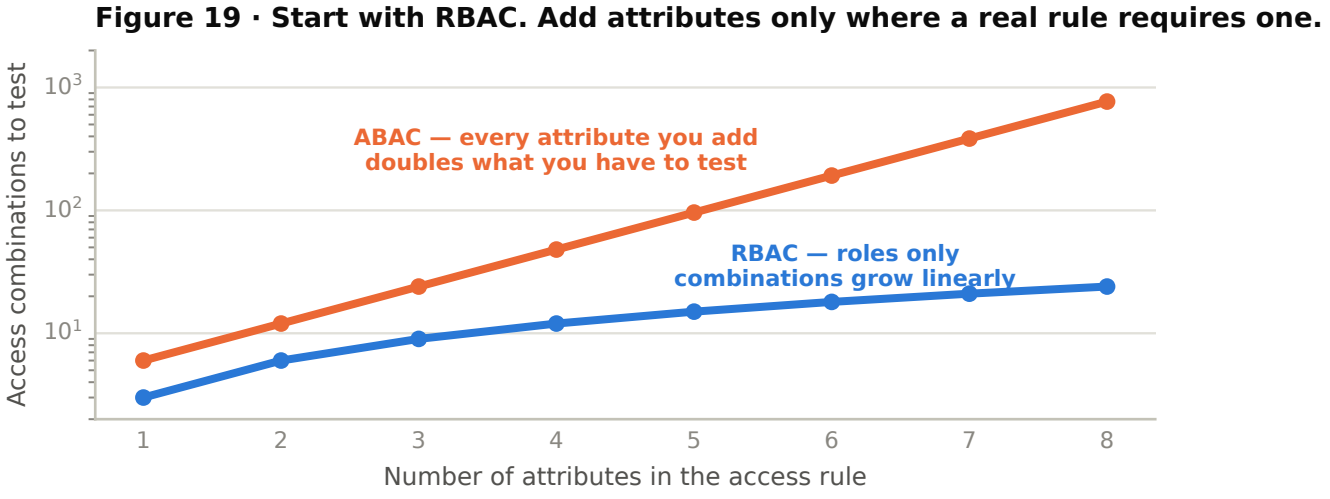

Figure 19. Every attribute you add multiplies the combinations you have to test. An access rule nobody can test is an access rule nobody should trust. Start with RBAC; add attributes only where a real rule requires one.

4.5 Secure tool calling — every safety property lives outside the model

Figure 21 · Every safety property lives outside the model, because the model cannot enforce any of them

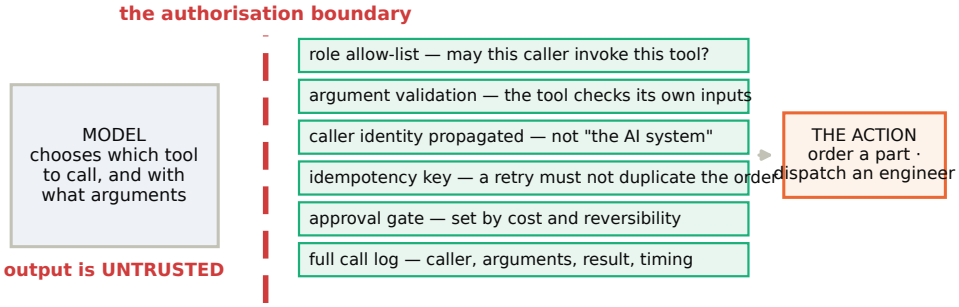

Figure 21. The model chooses the tool and the arguments; everything that makes the resulting action safe is enforced on the far side of a boundary the model cannot cross. Idempotency deserves particular attention: retries are normal, and a duplicated parts order is not.

> **The principle to repeat all week.** Treat model output as untrusted user input at every point where it crosses into another system. It is generated text, not a verified instruction.

4.6 Four leakage paths

Figure 20 · Four ways sensitive content leaves the system — check all four in your own code

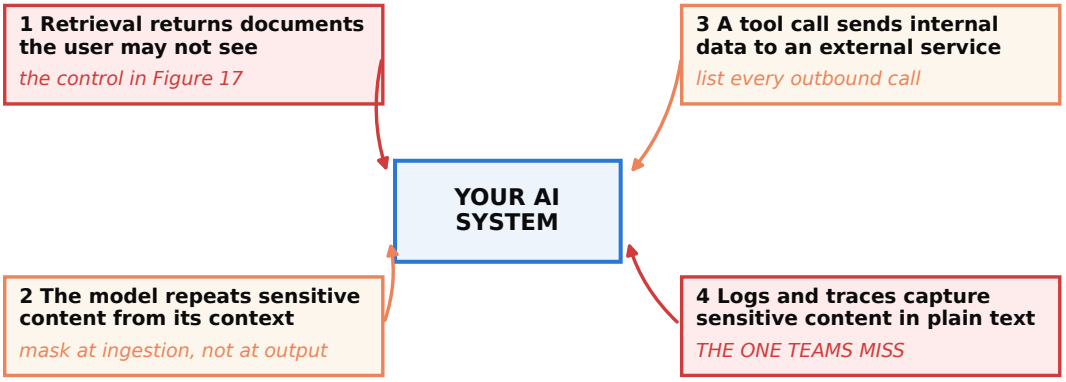

Figure 20. The fourth is the most commonly overlooked. Observability systems store full prompts and responses by default — that is what makes them useful — so your own trace log is frequently the least-protected copy of your most sensitive content.

> **Embeddings are not anonymised data.** An embedding is a lossy but information-rich representation of the source text, and partial reconstruction of original content from embeddings has been repeatedly demonstrated. Treat the vector store with the same data classification as the documents it was derived from — teams frequently apply weaker controls to it on the assumption that vectors are meaningless numbers.

> **Prompt injection — named here, attacked later in the week.** If your system reads content it did not author — a service ticket, a supplier document, an email — that content can contain instructions aimed at the model. The architectural defence is structural: instructions and data occupy different trust levels, and no tool call is ever authorised by something the model *read* rather than by who the caller *is*.

Appendix A — Checklists you can apply directly

Reference

The six selection questions, in order

1. What is the data type? documents · images · sensor streams · structured 2. Does ground truth exist? no labels rules out supervised learning entirely 3. What is the latency budget? ms → edge · seconds → cloud · hours → batch 4. Can it act, or only recommend? action requires approval, audit and rollback 5. Who is allowed to see what? changes the data architecture, not just the API 6. What must it integrate with? usually the largest hidden cost in the project

The nineteen layers

Access interface · API · authentication · authorisation · RBAC Reasoning orchestration · tool and MCP layer · model serving Data vector · relational · object · graph · feature store Governance prompt registry · Skill registry · model registry Operations monitoring · logging · audit trail Improvement feedback loop · human approval

Mark each as present, planned, or excluded with a reason. An exclusion with a reason is a complete answer.

The six orchestration responsibilities

routing · sequencing · retry with backoff · timeouts · context assembly · guardrail invocation

Every external call needs a deadline, and the deadline must be shorter than your caller's patience.

The eleven storage layers

raw · cleaned · features · embeddings · vector index · metadata ground truth · feedback · audit · model artifacts · prompts and policy

The eight data quality checks

completeness · validity at ingestion consistency · uniqueness · referential integrity at cleaning freshness · distribution continuous label quality periodic

Secure retrieval — the test

Ask a question whose answer exists ONLY in a restricted document, as a user who may not see it. answered → you have retrieve-then-filter. Not a control. refused → the access predicate is inside the query. Correct. A system that never returns "nothing available to you" has probably not implemented access control at all.

The four leakage paths

1. retrieval returns documents the user may not see 2. the model repeats sensitive content from its context 3. a tool call sends internal data to an external service 4. logs and traces capture sensitive content in plain text ← the one teams miss

Appendix B — Glossary

Term	Plain meaning
ABAC	Permissions computed from attributes of user, resource and context. Expressive, and hard to test.
Agent	A system that decides its own sequence of actions. Use when the steps genuinely cannot be enumerated.
Anomaly detection	Learns what normal looks like and flags departures. Needs no labelled failures.
Audit trail	Who did what, when, with which data, under whose authority. Immutable, long retention.
Authentication	Who is this? Distinct from authorisation.
Authorisation	What may they do? A user can be authenticated and still refused.
Batch inference	Scored on a schedule. Dramatically cheaper and simpler, and correct more often than teams assume.
Change data capture	Ingestion that reacts to changes in a source database. Risk: silent divergence.
Data contract	An agreement between producer and consumer, enforced automatically rather than assumed.
Digital twin	Simulation that generates training data reality cannot. Risk: learning the simulator's quirks.
Edge AI	Inference at the machine. Sub-100 ms and no network dependency, at the cost of model size and updates.
Embedding	Text as a list of numbers, positioned in a space a *particular model* built. Only meaningful relative to that model.
Event-driven ingestion	Triggered by an occurrence. The honest answer when a stale answer is actively harmful.
Feature store	One feature definition, used by both training and serving. Exists to prevent training–serving skew.
Fine-tuning	Changes the model's behaviour — style, format, narrow tasks. Not a way to add facts.
Human-in-the-loop	The system defers rather than guesses. An architectural component, not a fallback.
Idempotency	A state-changing call that is safe to retry. Retries are normal; a duplicated parts order is not.
Lineage	Where this data came from and every transformation applied to it.
Orchestration	The layer that turns a model call into a system. Routing, sequencing, retry, timeouts, context, guardrails.
Prompt registry	Prompts versioned, reviewed and tested like code, linked to the evaluation they produced.
RAG	Retrieval-augmented generation. Changes the information available, and produces citations.
RBAC	Permissions attached to roles. Simple, auditable, the right default.
Secure retrieval	The user's access rights are part of the retrieval query, so restricted documents are never candidates.
Skill registry	Reusable capability packages — instructions, schemas, tool bindings, safety rules, test cases.
Training–serving skew	A feature computed differently in training and in production. Silent, and a leading cause of unexplained underperformance.
Vector index	A searchable structure over embeddings. Rebuildable; must carry filterable metadata.
Workflow	A directed pipeline of known steps. Cheaper, faster, testable and predictable. Use it whenever you can list the steps.

> **If you remember five things.** The pattern is the decision with the longest half-life. Answer the six selection questions before building. A proof of concept has three components and a production system has nineteen. Access control belongs inside the retrieval query. And treat model output as untrusted input wherever it crosses into another system.